



PERTEMUAN 6

Array dalam Pemrograman Java

Mempelajari konsep array sebagai struktur data fundamental dalam Java — mulai dari deklarasi, penggunaan, penanganan kesalahan, hingga koleksi data yang lebih fleksibel.



Tipe Primitif vs Tipe Referensi

Tipe Primitif

Variabel yang menyimpan nilai secara langsung di memori. Contoh tipe primitif dalam Java:

- **int** — bilangan bulat
- **double** — bilangan desimal
- **boolean** — nilai benar/salah
- **byte, char, short, long, float**

Tipe Referensi

Semua tipe non-primitif adalah tipe referensi. Variabel referensi menyimpan *alamat memori* objek, bukan nilai itu sendiri. Kelas yang mendefinisikan tipe objek termasuk tipe referensi.

Contoh penggunaan:

```
Scanner input = new  
Scanner(System.in);  
int number =  
input.nextInt();
```



Apa Itu Array?

Array adalah sekelompok variabel yang disebut **elemen** atau **komponen**, yang semuanya memiliki tipe data yang sama dan disimpan secara berurutan di memori.

Koleksi Nilai Sejenis

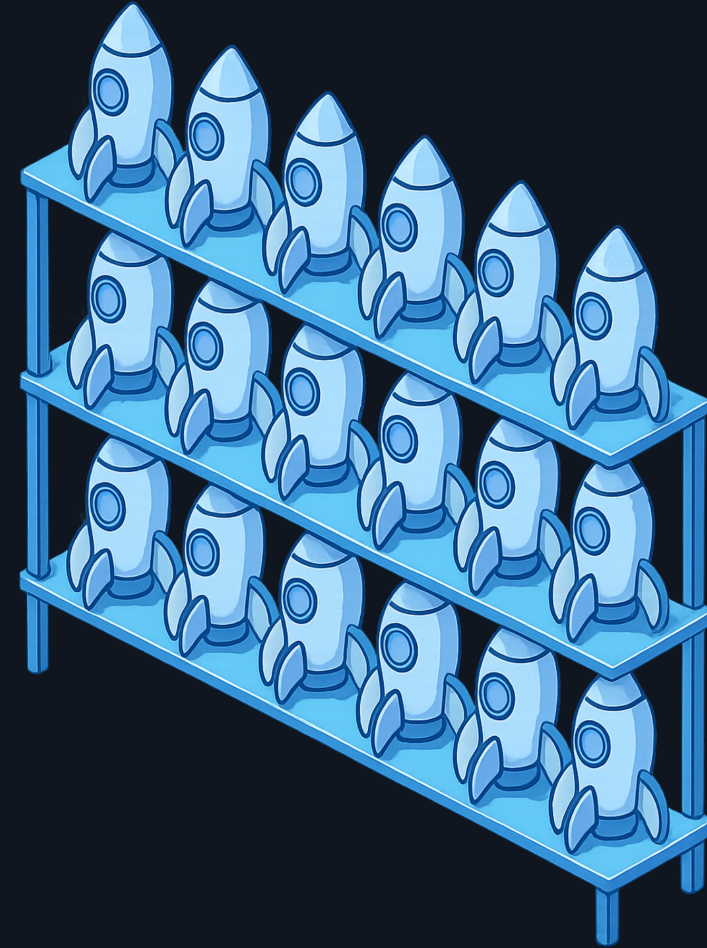
Semua elemen dalam sebuah array harus bertipe sama, misalnya semuanya `int` atau semuanya `String`.

Array adalah Objek

Dalam Java, array dianggap sebagai tipe referensi — bukan tipe primitif. Array dibuat di memori heap seperti objek biasa.

Akses via Indeks

Setiap elemen dapat diakses menggunakan indeks yang dimulai dari `0` hingga `panjang - 1`.



Deklarasi & Membuat Array

Objek array menempati ruang dalam memori. Seperti objek lainnya, array dibuat dengan keyword `new`. Kita perlu menentukan **tipe elemen** dan **jumlah elemen** saat membuat array.

Sintaks Dasar

```
// Deklarasi array integer dengan 12 elemen
int[] c = new int[12];

// Deklarasi array String dengan 100 elemen
String[] b = new String[100];

// Deklarasi array String dengan 27 elemen
String[] x = new String[27];
```

Penjelasan

- `int[]` — mendeklarasikan variabel array bertipe integer
- `new int[12]` — mengalokasikan memori untuk 12 elemen integer
- Elemen array integer secara default bernilai `0`
- Elemen array String secara default bernilai `null`

❗ Ukuran array bersifat tetap setelah dibuat dan tidak dapat diubah selama program berjalan.

Penggunaan Array — Bagian 1

1 Membuat dan Menginisialisasi Array

Proses mendeklarasikan array dan memberikan ukuran atau nilai awal agar dapat digunakan dalam program. Setelah array dideklarasikan, elemen-elemennya dapat diisi satu per satu melalui indeks.

```
int[] nilai = new int[5];  
nilai[0] = 90; nilai[1] = 85;
```

2 Menggunakan Array Initializer

Cara cepat untuk membuat dan mengisi array secara langsung pada saat deklarasi menggunakan tanda kurung kurawal {}, tanpa perlu menggunakan keyword `new` secara eksplisit.

```
int[] nilai = {90, 85, 78, 92, 88};
```



Penggunaan Array — Bagian 2

Menghitung Nilai untuk Disimpan dalam Array

Melibatkan perhitungan logika atau operasi matematika untuk menentukan nilai yang akan dimasukkan ke dalam elemen array. Nilai-nilai ini bisa berasal dari rumus, input pengguna, atau hasil operasi lainnya.

```
for (int i = 0; i < 5; i++) {  
    hasil[i] = i * i; // menyimpan kuadrat  
}
```

Menjumlahkan Elemen-elemen dalam Array

Teknik menjumlahkan semua nilai dalam array, biasanya menggunakan perulangan `for` untuk mengakses setiap elemen secara berurutan dan mengakumulasikan hasilnya ke variabel `total`.

```
int total = 0;  
for (int i = 0; i < nilai.length; i++) {  
    total += nilai[i];  
}
```

Penggunaan Array — Bagian 3



Diagram Batang dari Data Array

Data dalam array dapat divisualisasikan sebagai grafik batang untuk memudahkan pemahaman distribusi data. Setiap elemen array merepresentasikan satu batang dalam grafik yang ditampilkan di konsol atau antarmuka grafis.



Elemen Array sebagai Penghitung

Array digunakan untuk mencatat jumlah kejadian (frekuensi) dari suatu nilai atau kategori. Setiap indeks array mewakili satu kategori, dan nilainya menunjukkan berapa kali kategori itu muncul dalam data.

```
frekuensi[nilai]++;
```

Penggunaan Array — Analisis Survei

Salah satu penerapan paling praktis dari array adalah dalam menganalisis hasil survei. Data dari responden disimpan dalam array dan kemudian diproses untuk mendapatkan insight yang bermakna.

Menyimpan Jawaban

Setiap jawaban responden disimpan sebagai elemen array, misalnya nilai 1–5 untuk skala kepuasan.

Menghitung Frekuensi

Array frekuensi digunakan untuk menghitung berapa banyak responden yang memilih setiap pilihan jawaban.

Menyajikan Hasil

Hasil analisis ditampilkan sebagai laporan atau diagram untuk membantu pengambilan keputusan berbasis data.



Exception Handling dalam Java

Dalam pemrograman, kadang muncul situasi tak terduga saat program dijalankan — seperti kesalahan data atau logika. Penanganan kondisi seperti ini disebut **exception handling**, yang memungkinkan program tetap berjalan atau memberikan pesan kesalahan yang jelas.

→ Contoh Situasi Nyata

Dalam program survei mahasiswa, hasil masih bisa ditampilkan meskipun ada satu data yang tidak sesuai. Dalam kasus yang lebih berat, program harus dihentikan setelah memberitahu pengguna.

→ Sumber Kesalahan

Kesalahan bisa muncul dari sistem Java (seperti indeks array tidak valid) atau dibuat oleh programmer sendiri untuk situasi tertentu, guna membuat aplikasi lebih tangguh.



Exception Handling: Blok "try"

Untuk menangani pengecualian, tempatkan kode yang berpotensi menimbulkan kesalahan di dalam blok `try`. Blok ini "mencoba" menjalankan kode, dan jika terjadi kesalahan, eksekusi akan langsung berpindah ke blok `catch`.

Struktur Dasar

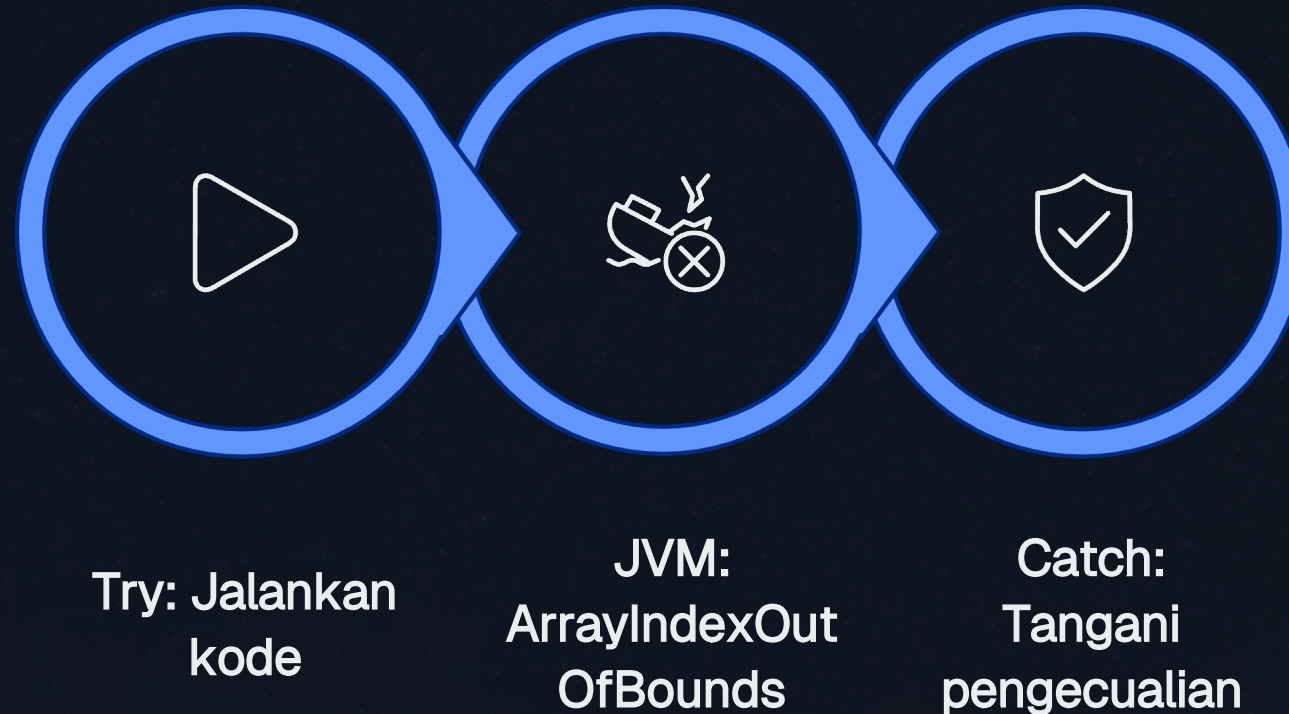
```
try {  
    // kode yang mungkin  
    // menimbulkan exception  
    int[] arr = new int[5];  
    arr[10] = 99; // index tidak valid!  
} catch (Exception e) {  
    // menangani exception  
    System.out.println(e.toString());  
}
```

Cara Kerja

- Blok `try` — berisi kode yang mungkin menimbulkan exception
- Blok `catch` — berisi kode yang menangani exception jika terjadi
- Jika tidak ada exception, blok `catch` dilewati sepenuhnya
- Program melanjutkan eksekusi setelah blok `catch`

Exception Handling: Blok "catch"

Ketika program menemukan nilai tidak valid dalam array, JVM menghasilkan sebuah pengecualian. Contoh: mengakses indeks yang tidak ada dalam array frekuensi (indeks 0–5) dengan nilai 14 akan menyebabkan `ArrayIndexOutOfBoundsException`.



Penting diingat: variabel lokal yang dideklarasikan di dalam blok `try` tidak dapat diakses di dalam blok `catch` karena sudah berada di luar cakupan (*scope*) saat exception terjadi.

Exception Handling: Metode "toString"

Menampilkan Pesan Kesalahan

Saat sebuah kesalahan tertangkap dalam blok `catch`, program dapat menampilkan pesan yang menjelaskan masalah tersebut. Ini dilakukan dengan memanggil metode `toString()` dari objek `exception` secara otomatis.

```
catch (ArrayIndexOutOfBoundsException e) {  
    System.out.printf(  
        "%s%n%n", e.toString()  
    );  
}
```

Alur Setelah Penanganan

Setelah pesan kesalahan ditampilkan, sistem menganggap masalah sudah ditangani. Program kemudian **melanjutkan eksekusi** ke bagian kode setelah blok `catch` — dalam kasus survei, perulangan tetap berjalan ke iterasi berikutnya tanpa program dihentikan.

- ✅ Exception handling membuat program lebih **robust** (tangguh) dan memberikan pengalaman pengguna yang lebih baik.

Enhanced For Loop (For-Each)

Perulangan *enhanced for* adalah bentuk perulangan yang dirancang khusus untuk menelusuri elemen-elemen dalam array atau koleksi, tanpa perlu membuat penghitung (indeks) secara manual.

Perulangan Biasa (for)

```
int[] nilai = {90, 85, 78};  
for (int i = 0; i <  
    nilai.length; i++) {  
  
    System.out.println(nilai[i]);  
}
```

Enhanced For Loop (for-each)

```
int[] nilai = {90, 85, 78};  
for (int n : nilai) {  
    System.out.println(n);  
}
```

Lebih Sederhana

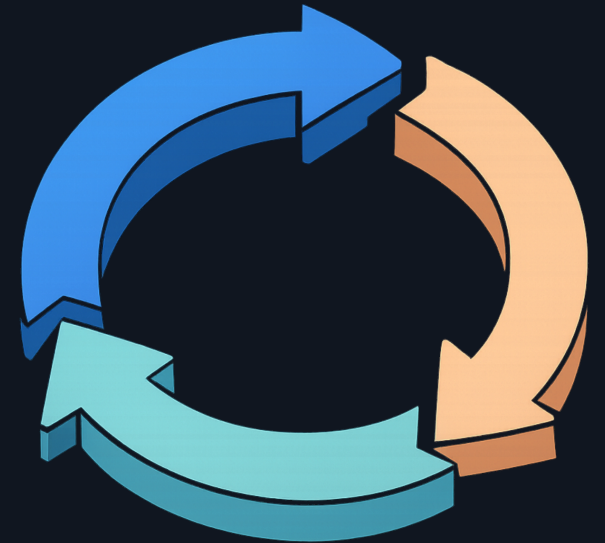
Kode lebih ringkas tanpa perlu mengelola variabel indeks.

Lebih Aman

Risiko mengakses indeks di luar batas array dihindari secara otomatis.

Lebih Mudah Dibaca

Niat kode lebih jelas: "untuk setiap elemen dalam array, lakukan ini."



Passing Arrays ke Methods

Bagian ini menunjukkan cara meneruskan array dan elemen array individual sebagai argumen ke metode. Untuk meneruskan array ke metode, cukup tentukan **nama array tanpa tanda kurung siku**.

Contoh Deklarasi & Pemanggilan

```
// Deklarasi array
double[] hourlyTemperatures =
    new double[24];

// Memanggil method dengan array
modifyArray(hourlyTemperatures);

// Memanggil method dengan satu elemen
modifyElement(hourlyTemperatures[3]);
```

Perbedaan Penting

- Saat `modifyArray(arr)` dipanggil, seluruh array diteruskan — perubahan pada elemen array **terlihat** dari luar metode.
- Saat `modifyElement(arr[3])` dipanggil, hanya nilai satu elemen yang dikirim sebagai *copy* — perubahan di dalam metode **tidak memengaruhi** nilai aslinya.



Pass-By-Value vs Pass-By-Reference

Dalam banyak bahasa pemrograman, ada dua cara umum untuk mengirim data ke dalam metode. Memahami perbedaannya penting agar tidak terjadi bug yang sulit dilacak.

Pass-By-Value

Yang dikirim ke metode adalah **salinan** dari nilai asli. Perubahan di dalam metode **tidak memengaruhi** nilai aslinya di luar metode. Ini seperti memberikan fotokopi dokumen — perubahan pada fotokopi tidak mengubah dokumen asli.

Pass-By-Reference

Yang dikirim adalah **alamat atau referensi** ke data asli. Perubahan di dalam metode **bisa memengaruhi** data aslinya. Ini seperti memberikan kunci ke lemari dokumen — perubahan di dalam langsung mengubah dokumen asli.

Java: Hanya Pass-By-Value

Java hanya mendukung **pass-by-value**. Semua data yang dikirim ke metode adalah salinan — baik nilai primitif maupun referensi ke objek. Namun, perilakunya berbeda tergantung tipe datanya.

1 Tipe Primitif (int, double, dll)

Perubahan nilai di dalam metode **tidak akan mengubah** nilai aslinya di luar metode. Contoh: jika `array[3]` dikirim ke metode dan digandakan di dalamnya, nilai `array[3]` di luar metode tetap tidak berubah.

2 Objek & Array (Referensi)

Yang dikirim adalah **salinan referensi** — bukan objeknya. Metode bisa mengubah *isi* objek atau array, dan perubahan itu akan terlihat dari luar. Namun, metode tidak dapat membuat referensi menunjuk ke objek baru.

Dimensi Array

Array dalam Java tidak terbatas pada satu dimensi. Java mendukung array multidimensi yang berguna untuk merepresentasikan data dalam bentuk tabel, matriks, atau struktur yang lebih kompleks.

1

Array Satu Dimensi

Array linear paling dasar. Setiap elemen diakses dengan satu indeks. Cocok untuk daftar nilai berurutan seperti nilai mahasiswa atau data harian.

```
int[] angka = {1, 2, 3, 4, 5};
```

2

Array Dua Dimensi

Mewakili tabel dengan baris dan kolom. Setiap elemen diakses dengan dua indeks: baris dan kolom. Cocok untuk data matriks atau jadwal.

```
int[][] matrix = new int[3][4];
```

3

Array dengan Baris Berbeda Panjang

Java mendukung *jagged arrays* — array dua dimensi di mana setiap baris dapat memiliki jumlah kolom yang berbeda, memberikan fleksibilitas dalam penggunaan memori.

```
int[][] jagged = new int[3][];  
jagged[0] = new int[2];  
jagged[1] = new int[4];
```

Collections dan Class ArrayList

Java menyediakan sekumpulan struktur data siap pakai yang disebut *collections* untuk menyimpan kumpulan objek yang saling berkaitan di memori. Kelas-kelas ini menawarkan cara yang efisien untuk mengatur, menyimpan, dan mengambil data.

Keunggulan Collections

- Mempercepat proses pengembangan aplikasi
- Tidak perlu membuat struktur data dari awal
- Fleksibilitas ukuran — bisa bertambah atau berkurang secara dinamis
- Dilengkapi metode bawaan untuk pengurutan, pencarian, dan manipulasi

Array vs Collections

Array memiliki ukuran tetap yang tidak bisa berubah setelah dibuat.

Collections, seperti `ArrayList`, menawarkan fleksibilitas lebih luas — ukurannya dapat berubah secara dinamis sesuai kebutuhan program.

```
import java.util.ArrayList;  
ArrayList<String> nama =  
    new ArrayList<>();
```



Operasi Dasar ArrayList

Kelas `ArrayList` menyediakan berbagai metode bawaan yang memudahkan pengelolaan data secara dinamis. Berikut adalah operasi-operasi utama yang sering digunakan:



Menambahkan Elemen — `add()`

Menambahkan elemen baru ke akhir list, atau di posisi indeks tertentu. Ukuran `ArrayList` bertambah secara otomatis.

```
nama.add("Budi");  
nama.add(0, "Ani");
```



Mengambil Elemen — `get()`

Mengakses elemen pada indeks tertentu, mirip dengan akses array menggunakan tanda kurung siku.

```
String s = nama.get(0);
```



Menghapus Elemen — `remove()`

Menghapus elemen berdasarkan indeks atau nilai objek. Elemen-elemen setelahnya secara otomatis bergeser ke kiri.

```
nama.remove(1);  
nama.remove("Budi");
```



Ukuran List — `size()`

Mengembalikan jumlah elemen yang saat ini tersimpan dalam `ArrayList`. Berbeda dengan `length` pada array biasa.

```
int jumlah = nama.size();
```

Terima Kasih!

Semoga materi **Array dalam Java** pada Pertemuan 6 ini bermanfaat dan mudah dipahami.
Terus berlatih dan eksplorasi konsep-konsep yang telah dipelajari!

Array Fundamentals

Tipe primitif vs referensi, deklarasi, inisialisasi, dan penggunaan dasar array.

Exception Handling

Blok try-catch untuk menangani kesalahan runtime seperti `ArrayIndexOutOfBoundsException`.

For-Each & Methods

Enhanced for loop, passing arrays ke methods, dan konsep pass-by-value Java.

Collections

`ArrayList` sebagai alternatif array yang dinamis dan fleksibel untuk mengelola data.

